VERIQTA

GitHub Actions: Your First Pipeline Step by Step



Code → Build → Test → Deploy → Monitor

- Understand GitHub Actions
- Build your first workflow
- Run automated tests
- Create and upload artifacts
- Use secrets and permissions
- Speed up your pipeline
- Handle failures and troubleshoot
- Run CI on pull requests
- Protect your main branch
- Build a complete working pipeline

Instagram @veriqta
GitHub @veriqta
LinkedIn @veriqta
Telegram @veriqta
X @veriqta
Discord @veriqta

VERIQTA

What Is GitHub Actions?

1. What Is GitHub Actions?

GitHub Actions is an **automation service** built into **GitHub** that lets you run **workflows** directly in your repository. It automatically performs tasks such as checking code, running tests, building applications, and deploying to servers.



GitHub Actions
Automate • Test • Build • Deploy

2. Manual Checks vs Automated Checks

Manual Checks (Without GitHub Actions)

- ✗ You run tests on your own computer
- ✗ You check if the code works manually
- ✗ It can be slow and easy to forget a step
- ✗ Different team members may test differently
- ✗ Bugs can be missed
- ✗ Code may be merged even if tests fail

Automated Checks (With GitHub Actions)

- ✓ Runs automatically when you push code
- ✓ Uses a clean and consistent environment
- ✓ Runs the same tests every time
- ✓ Catches problems early
- ✓ Gives clear results in the repository
- ✓ Helps keep the main branch stable and safe

3. What Is Continuous Integration (CI)?

Continuous Integration (CI) is the practice of automatically checking code every time a change is pushed to a repository. It helps the team find and fix problems early, before the code is merged into the main branch.

Key Idea
CI runs automated checks on every change so you know quickly if the code is working.

4. How It Works



5. Real-World Connection

Teams use GitHub Actions to make sure new changes do not break the application. It helps keep the **main branch stable**, improves **code quality**, and reduces the risk of deploying broken code to production. Many companies use CI with GitHub Actions as part of their DevOps and software development process.

Why It Matters
Automated checks build trust, save time, and help teams deliver better and more reliable software.

How a GitHub Actions Pipeline Actually Works

1. Key Components

Event
- A specific activity that triggers a workflow.
- Examples: push, pull_request, workflow_dispatch.
- Tells GitHub when to start the pipeline.

Workflow
- A YAML file that defines the pipeline.
- Contains one or more jobs.
- Stored in .github/workflows/
- Runs when an event occurs.

Job
- A set of steps that run on the same runner.
- Jobs can run in parallel.
- Each job has its own runner.
- Used to organize work (e.g. test, build, deploy).

Runner
- A virtual machine that runs your jobs.
- Provided by GitHub (e.g. ubuntu-latest, windows-latest, macos-latest).
- Has a fresh, temporary environment for each job.

Step
- An individual task inside a job.
- Steps run sequentially (one after another).
- Each step can run a command or use an action.

Action
- A reusable piece of code that performs a specific task.
- Examples: actions/checkout, actions/setup-python.
- An action is used inside a step.
- It is not an entire workflow.

2. How They Fit Together

Push Event → **Workflow** → **Job** → **Runner** → **Steps**

You push code to the repository.

GitHub reads the workflow YAML file.

A job is created and assigned to a runner.

A GitHub-hosted runner runs the job in a clean environment.

Steps run one by one and use actions to complete tasks.

3. Jobs Run in Parallel, Steps Run Sequentially

Jobs can run in parallel (at the same time)

- Job 1 Lint Code
- Job 2 Run Tests
- Job 3 Build Package

All jobs run at the same time on separate runners

Steps run sequentially (one after another)

Steps inside a job

1. Checkout code
2. Set up environment
3. Run tests
4. Upload artifact

Steps in a job always run in order.

The next step starts only after the previous step finishes.

Beginner Mistake

Confusing an action with an entire workflow.

- An action is a single, reusable task (e.g. actions/checkout).
- A workflow is the complete pipeline defined in a YAML file.
- A workflow contains jobs, and each job contains steps that use actions.

Remember This

- Events trigger workflows.
- Workflows contain jobs.
- Jobs run in parallel.
- Steps inside a job run sequentially.
- Steps use actions to perform tasks.

Prepare Your First CI Project

1. Create a GitHub Repository

1. Go to GitHub and click New repository.
2. Give it a name (e.g. python-ci-app).
3. Choose Public or Private.
4. Do not initialize with a README (we will add our own files).
5. Click Create repository.



2. Project Files and Their Purpose

app.py

- A simple Python application.
- Contains the code we want to test in the pipeline.
- Keeps the example small and focused.

test_app.py

- Contains automated tests for the application.
- We will use pytest to run these tests in GitHub Actions.
- Helps catch bugs early.

requirements.txt

- Lists the Python dependencies needed for the project.
- The pipeline will install these dependencies before running tests.
- Keeps the environment consistent.

README.md

- Project description and instructions.
- Explains what the application does.
- Helps other developers understand the project.
- Commonly used in real repositories.

3. Repository Structure

```
python-ci-app/
├── .github/
│   └── workflows/
│       └── ci.yml          # GitHub Actions workflow file
├── app.py                  # Python application code
├── test_app.py             # Automated tests (pytest)
├── requirements.txt        # Python dependencies
└── README.md               # Project description
```

Where Do GitHub Actions Files Go?

- All workflow files are stored in .github/workflows/ inside your repository.
- Each workflow file is a YAML file (e.g. ci.yml).
- You can have multiple workflow files in the same folder.

Important

- If the .github/workflows/ folder does not exist, create it.
- You can create the folder and file locally or directly on GitHub.

4. Practice: Create the Project and Push the First Commit

1. Create a folder on your computer and add the project files.
2. Initialize Git and push the code to your GitHub repository.
3. Confirm that the files appear on GitHub.

Example Commands (run in your terminal)

```
mkdir python-ci-app
cd python-ci-app
git init
git add .
git commit -m "Initial commit: add project files"
git branch -M main
git remote add origin https://github.com/<your-username>/python-ci-app.git
git push -u origin main
```

Write Your First Workflow YAML

1. What Is YAML?

- YAML (YAML Ain't Markup Language) is a simple and human-readable format used to configure GitHub Actions.
- It tells GitHub what to do, when to do it, and how to do it.
- YAML uses indentation to define structure, so spacing is important.

Why It Matters

You use YAML to define your CI pipeline. GitHub reads this file and runs the steps for you.

2. Create the Workflow File

- In your repository, create a folder named .github/workflows/
- Create a file named ci.yml inside it.



3. Example Workflow: ci.yml

.github/workflows/ci.yml

```yaml
name: My First Workflow
on:
  push:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Print a message
        run: echo "Hello from GitHub Actions!"
```

4. Understand the Main Parts

name
- A friendly name for your workflow.
- Appears in the GitHub Actions tab.

on
- Defines when the workflow runs.
- Common events: push, pull_request, workflow_dispatch.

jobs
- A workflow can have one or more jobs.
- Each job runs on a separate runner.

runs-on
- Specifies the type of GitHub-hosted runner.
- Example: ubuntu-latest.

steps
- A list of tasks that run in order inside a job.
- Each step can run a command or use an action.

run
- Runs a shell command on the runner.
- Used for simple tasks like installing dependencies or printing a message.

5. Practice: Run Your First Workflow

1. Create the .github/workflows/ci.yml file with the example above.
2. Commit the file to your repository.
3. Push the commit to the main branch.
4. Go to the Actions tab in GitHub.
5. You should see your workflow run and a message like "Hello from GitHub Actions!" in the logs.

Expected Result

The workflow runs successfully and prints:

```
Hello from GitHub Actions!
```

Trigger Your First Pipeline

1. Understand Workflow Events

A workflow starts when a specific **event** happens in your repository.
The event tells GitHub when to run your pipeline.

push

- Runs when you push code to a branch (e.g. main).
- Used for continuous integration.
- Common for running tests whenever you change code.

When to use:
Run CI checks on every push to keep your code working.

pull_request

- Runs when a pull request is opened, updated or reopened.
- Checks the changes before merging.
- Helps catch problems early during code review.

When to use:
Run tests and checks on pull requests before merging into main.

workflow_dispatch

- Runs only when you manually start it from the GitHub Actions tab.
- Useful for testing, re-running workflows, or manual tasks (e.g. building a release).

When to use:
Run a workflow manually when needed.

2. Example: Use These Events in Your Workflow

.github/workflows/ci.yml

```yaml
name: My First Pipeline
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
  workflow_dispatch:
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Print a message
        run: echo "Hello from GitHub Actions!"
```

What This Means

- The workflow runs when you push to main.
- It also runs when a pull request targets main.
- You can also run it manually from the Actions tab.

3. Workflow Flow



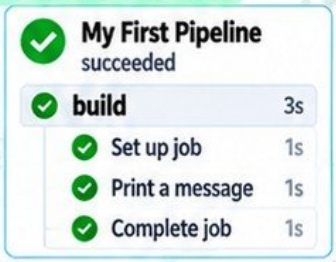
4. Trigger the First Pipeline by Pushing a Commit

1. Make a change to a file (e.g. update README.md).
2. Commit the change.
3. Push the commit to the main branch.
4. This triggers the workflow using the push event.

5. Check the Workflow Run

1. Go to your repository.
2. Click the Actions tab.
3. Select the latest run.
4. You will see the workflow name, job, steps and their status.

6. Verify Success



Beginner Mistake

Thinking a workflow only runs on push. There are multiple events, and you can use one or more depending on your need.

Remember This

Events control when a workflow runs.
You can use push, pull_request, workflow_dispatch, or a combination of them.

Understand GitHub-Hosted Runners

1. What Is a Runner?

- A runner is a **virtual machine** that runs your workflow jobs.
- When a workflow starts, GitHub provides a **clean runner** to execute your jobs.
- The runner runs the steps in your job and then is **destroyed** after the job completes.

2. Why Jobs Need a Runner

- Jobs contain steps that run commands or use actions.
- Each job needs a runner to execute those steps.
- You do not need to set up your own server. GitHub provides the runner for you.
- The runner has the operating system and common tools already installed.



3. ubuntu-latest Runner

- **ubuntu-latest** is a GitHub-hosted runner that uses the latest stable version of Ubuntu Linux.
- It comes with many common tools already installed (e.g. Git, Python, Node.js, Docker, etc.).
- GitHub keeps it updated with the latest security patches.

ubuntu-latest

- ✅ Ubuntu Linux
- ✅ Common tools pre-installed
- ✅ Always up to date
- ✅ Clean environment every time

4. Temporary Environment

- Each runner is **temporary and fresh.**
- Your files and changes exist only during the job run.
- Nothing you install on the runner is saved for the next run.
- This ensures clean, repeatable and safe build environments.

⚠ **Locally installed tools on your computer** do not exist on the runner. You must install any extra tools or dependencies in your workflow.

5. Example: Inspect the Runner Environment

Add these steps to your workflow to see information about the runner and installed tools.

.github/workflows/ci.yml

```yaml
jobs:
  check-runner:
    runs-on: ubuntu-latest
    steps:
      - name: Show runner OS
        run: |
          echo "OS Information:"
          cat /etc/os-release
          echo "\nPython Version:"
          python --version
          echo "\nAvailable Tools:"
          git --version
```

Example Output (in GitHub Actions Logs)

```
OS Information:
NAME="Ubuntu"
VERSION="22.04.4 LTS (Jammy Jellyfish)"
...

Python Version:
Python 3.11.6

Available Tools:
git version 2.43.0
```

Common Tools on ubuntu-latest

- ✅ Git
- ✅ Python
- ✅ Node.js
- ✅ Docker
- ✅ curl
- ✅ wget

and many more....

7. Production Connection

- Using consistent, clean and up-to-date runners helps your team build reliable pipelines.
- It reduces "works on my machine" problems.
- Repeatable build environments make debugging easier and ensure your code is tested in a predictable setup.

8. Key Takeaways

- A runner is a temporary virtual machine provided by GitHub.
- ubuntu-latest is a popular Linux runner with many tools.
- You must install any extra tools or dependencies.
- Each run starts with a clean environment.
- Consistent runners help you build reliable and repeatable pipelines.

Check Out Your Repository Code

1. What Is actions/checkout?

actions/checkout

- actions/checkout is an official GitHub Action that downloads your repository code into the runner's workspace.
- It makes your project files available to the steps that run after it.
- You must add this step before you can use your application files.

2. Why Your Job Does Not Have the Code Automatically

- When a job starts, the runner is a fresh virtual machine.
- It does not contain your repository files.
- GitHub keeps your code in a separate storage system.
- actions/checkout is the step that copies your code from GitHub to the runner's workspace.



3. Add actions/checkout to Your Workflow

.github/workflows/ci.yml

```yaml
name: Checkout Example
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Check out repository code
        uses: actions/checkout@v4

      - name: Show files in workspace
        run: |
          pwd
          ls -la
```

4. What Happens After Checkout?

- Your repository files are downloaded to the runner.
- The files are placed in the runner's workspace.
- Subsequent steps can now access and use your application code, tests and other files.

5. Verify the Files Are Available

Example Output (GitHub Actions Logs)

```
/home/runner/work/python-ci-app/python-ci-app
total 24
drwxr-xr-x  3 runner runner 4096 Oct 14 12:00 .
drwxr-xr-x  3 runner runner 4096 Oct 14 12:00 ..
drwxr-xr-x  2 runner runner 4096 Oct 14 12:00 .github
-rw-r--r--  1 runner runner  123 Oct 14 12:00 app.py
-rw-r--r--  1 runner runner  234 Oct 14 12:00 test_app.py
-rw-r--r--  1 runner runner   56 Oct 14 12:00 requirements.txt
-rw-r--r--  1 runner runner  433 Oct 14 12:00 README.md
```

6. Common Failure Without Checkout

- If you try to run your application files before using actions/checkout, the files will not be found.
- You will get errors like "No such file or directory".

Example Error (without checkout)

```
Run python app.py
python: can't open file '/home/runner/work/app.py':
[Errno 2] No such file or directory
```

7. Correct Order of Steps

1. Check out the repository code (actions/checkout).
2. Verify the files are present (pwd, ls -la).
3. Install dependencies (if needed).
4. Run your application or tests.

8. Key Takeaways

- A job does not automatically have your repository files.
- Use actions/checkout to download the code.
- After checkout, your files are available in the runner's workspace.
- Always run checkout before trying to use your application files.

9. Production Connection

- Checkout ensures every run uses the exact code from your repository.
- This makes builds consistent and reproducible.
- Teams can trust that the code being tested is the same as what was pushed to GitHub.

Set Up Python and Install Dependencies

1. What Is actions/setup-python?

actions/setup-python

- **actions/setup-python** is an official GitHub Action that installs the specified version of Python on the runner.
- It makes the Python version available for your workflow steps.
- You can choose the exact Python version your project needs.

2. Why Specify the Python Version?

- Runners come with multiple versions of Python.
- The default version may change over time.
- Using a specific version ensures your pipeline is consistent and peproducible.
- It helps avoid compatibility issues between different Python versions.

Recommended Practice

Always specify the Python version your project uses (e.g. 3.11 or 3.12).

3. Add Python Setup and Install Dependencies

.github/workflows/ci.yml

```yaml
name: Python CI
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Check out code
        uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - name: Install dependencies
        run: python -m pip install -r requirements.txt
```

4. Verify Python Version and Dependencies

Example Commands (add as workflow steps)

```yaml
- name: Verify Python version
  run: |
    python --version
    python -m pip --version
- name: List installed packages
  run: |
    python -m pip list
```

Example Output (GitHub Actions Logs)

```
Python 3.11.6
pip 23.2.1 from /opt/hostedtoolcache/.../pip

Package       Version
-----------   -------
pip           23.2.1
pytest        7.4.3
requests      2.31.0
```

5. Application Dependencies vs Runner Tools

Application Dependencies

- These are the Python packages your application needs (listed in requirements.txt).
- Examples: requests, pytest, flask, etc.
- You install them in your workflow using:

`python -m pip install -r requirements.txt`

- They are specific to your project.

Runner Tools

- These are tools already available on the GitHub-hosted runner.
- Examples: Git, Docker, curl, Node.js, some Python versions, etc.
- You do not install these (unless you need a different version).
- They are provided by GitHub and may change over time.

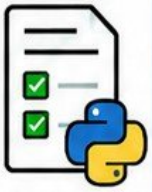
6. Key Takeaways

- Use actions/setup-python to install Python on the runner.
- Always specify the Python version.
- Install your project dependencies using requirements.txt.
- Verify the Python version and installed packages.
- This ensures a consistent and reproducible build environment.

Run Your First Automated Test

1. What Are Automated Tests?

- Automated tests are programs that check if your code works as expected.
- They run automatically in your pipeline whenever you push code or create a pull request.
- They help catch bugs early and prevent broken code from being merged.
- In CI, tests give you fast feedback and increase confidence in your changes.



Automatic checks for your code

2. Create a Simple Python Test with pytest

app.py

```python
def add(a, b):
    return a + b
```

A simple function that adds two numbers.

test_app.py

```python
from app import add

def test_add():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0
```

A test file that uses pytest to check that the function works correctly.

3. Add a Test Step to Your Workflow

.github/workflows/ci.yml

```yaml
name: Python CI
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Check out code
        uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - name: Install dependencies
        run: python -m pip install -r requirements.txt
      - name: Run tests
        run: python -m pytest -q
```

4. Run pytest in the Pipeline

- The command python -m pytest -q runs all tests.
- The -q flag shows a shorter output.
- If all tests pass, the step succeeds.
- If any test fails, the step fails and the workflow stops.

5. Understand Exit Codes

Exit code 0

- Means the command succeeded.
- All tests passed.
- The step is marked as successful.

Nonzero exit code (e.g. 1)

- Means the command failed.
- Usually happens when a test fails.
- The step is marked as failed and the workflow stops (unless configured otherwise).

6. Example Test Output (Pass)

GitHub Actions Logs (Tests Passed)

```
Run python -m pytest -q
============================= test session starts ==============================
platform linux -- Python 3.11.6, pytest-7.4.3
collected 1 item

test_app.py .                                                            [100%]

1 passed in 0.12s
```

7. Example Test Output (Fail)

GitHub Actions Logs (Tests Failed)

```
Run python -m pytest -q
============================= test session starts ==============================
collected 1 item

test_app.py F                                                            [100%]

=================================== FAILURES ===================================
_____________________________ test_add _____________________________
E       assert 4 == 5
E        +  where 4 = add(2, 3)
1 failed in 0.10s
```

8. Verify in GitHub Actions

1. Push your changes to the repository.
2. Go to the Actions tab in GitHub.
3. Open the latest workflow run.
4. Confirm that the test step completed successfully.

9. Key Takeaways

- Automated tests run in CI to catch bugs early.
- Use pytest to write and run Python tests.
- A zero exit code means success; a nonzero exit code usually means a failure.
- Passing tests give you confidence to merge your changes.

Break the Pipeline and Investigate the Failure

1. Intentionally Break the Application

Change the application so that the test will fail.

app.py (broken version)

```
def add(a, b):
    return a - b  # should be a + b
```

⚠ We changed the function to subtract instead of adding. This will make the existing test fail.

2. Push the Change and Observe the Failed Workflow

1. Commit the change: `git add .`
2. `git commit -m "Break the test"`
3. `git push`

▶ Pushing the change triggers the workflow. The tests will run automatically and fail.

3. View the Failed Workflow in GitHub

1. Go to your repository on GitHub.
2. Click the Actions tab.
3. Select the latest workflow run (with a red ✗).
4. Click the job to see the steps.
5. Click the failed step (e.g. Run tests) to view the error output.



4. Investigate the Error Output

```
Run python -m pytest -q
================== test session starts ==================
collected 2 items

test_app.py F.                                      [50%]

==================== FAILURES ====================
_____________________ test_add _____________________
def test_add():
    assert add(2, 3) == 5
E   assert -1 == 5
E    + where -1 = add(2, 3)

1 failed, 1 passed in 0.08s
```

5. Understand the Type of Failure

✗ Failing Test

- The pipeline ran correctly.
- The code executed but a test assertion failed.
- This means your application behavior is not what the test expects.
- You need to fix the code (or the test if the test is wrong).

⚙ Pipeline Configuration Error

- The workflow failed before or during setup.
- Caused by issues in the YAML file (e.g. wrong action name, invalid syntax, missing dependency, or wrong Python version).
- The code might not even run in this case.
- You need to fix the workflow configuration.

6. Fix the Code and Rerun the Checks

app.py (fixed version)

```
def add(a, b):
    return a + b  # correct implementation
```

1. Save the change.
2. Commit: `git commit -m "Fix add function"`
3. Push: `git push`
4. The workflow will run again and the tests should pass.

7. Successful Workflow After the Fix

```
Run python -m pytest -q
================== test session starts ==================
collected 2 items
test_app.py ..                                      [100%]
2 passed in 0.05s
```

8. Key Takeaways

- A failing test means the code ran but did not meet the expected result.
- A pipeline configuration error means the workflow setup is wrong.
- Always read the error message carefully to find the real cause.
- Fix the issue, push a new commit and let the pipeline run again.
- This process helps you catch and fix problems early.

Add Code Quality Checks Before Testing

1. Why Check Code Quality?

- Your code can work correctly but still have maintainability problems.
- Issues like unused imports, bad style, complex code or potential bugs can make the code harder to maintain.
- Automated code quality checks catch these simple defects early before code review and before testing.



Cleaner code is easier to maintain.

2. What Is Ruff?

Ruff

- Ruff is a fast Python linter that checks your code for style issues, bugs and maintainability problems.
- It is easy to use and works well in CI.
- It finds issues like unused imports, undefined variables and other common problems.
- We will use: `ruff check .`

- Fast
- Modern
- Works well in CI
- Helps keep your code clean

3. Add Ruff to Your Workflow

.github/workflows/ci.yml

```yaml
name: Python CI
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Check out code
        uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - name: Install dependencies
        run: python -m pip install -r requirements.txt
      - name: Run Ruff (linting)
        run: ruff check .
      - name: Run tests
        run: python -m pytest -q
```

4. Example: Add an Unused Import (Cause a Lint Failure)

app.py (with an unused import)

```python
import os  # unused import (will be flagged)

def add(a, b):
    return a + b
```

⚠ The code still works, but Ruff will report the unused import.

5. Linting Failure vs Test Failure

Ruff Linting Failure

```
> Run ruff check .
app.py:1:8: F401 `os` imported but unused
1 | import os
  |        ^^
Found 1 error.
```

- This is a code quality issue.
- The code might still run correctly.
- It means the code has maintainability problems (e.g. unused import).
- This step fails before tests run.

Pytest Test Failure

```
> Run python -m pytest -q
F    test_app.py:5: AssertionError
     def test_add():
>        assert add(2, 3) == 5
E        assert 4 == 5
```

- This is a failing test.
- The code executed but did not produce the expected result.
- It means there is a problem with your application logic.
- This step fails during testing.

6. Fix the Issue and Rerun

app.py (fixed)

```python
def add(a, b):
    return a + b  # removed unused import
```

1. Remove the unused import.
2. Save the change.
3. Commit: `git commit -m "Fix lint issue"`
4. Push: `git push`
5. The workflow runs again and the Ruff check passes.

7. DevOps Connection

- Catching simple defects early makes your codebase cleaner and easier to maintain.
- It reduces back-and-forth during code review.
- It improves code quality across the team.
- Automated linting in CI enforces consistent standards.
- This helps you build reliable and professional software over time.

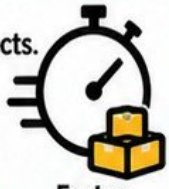
8. Key Takeaways

- Use Ruff to check your code quality: `ruff check .`
- Linting fails on maintainability issues; tests fail on logic errors.
- Fix the issue and rerun the workflow.
- Code quality checks help you catch problems early and keep your project healthy.

Speed Up Your Pipeline With Dependency Caching

1. Why Use Dependency Caching?

- Installing dependencies on every run can be **slow**, especially for larger projects.
- Caching stores a copy of your dependencies so they can be **reused** in future runs.
- This **reduces pipeline execution time** and saves compute resources.



Faster pipelines with caching

2. How Caching Works with actions/setup-python

- **actions/setup-python** can automatically cache Python dependencies.
- You specify the cache type (**pip**) and the dependency file (e.g. **requirements.txt**).
- GitHub creates a cache key based on:
  - The operating system (OS)
  - The Python version
  - The content (hash) of the dependency file



actions/ setup-python

3. Add Caching to Your Workflow

.github/workflows/ci.yml

```yaml
name: Python CI
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Check out code
        uses: actions/checkout@v4
      - name: Set up Python (with caching)
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
          cache: 'pip'
          cache-dependency-path: requirements.txt
      - name: Install dependencies
        run: python -m pip install -r requirements.txt
      - name: Run tests
        run: python -m pytest -q
```

4. Understanding Cache Keys

Cache key is based on:

- Runner OS (e.g. ubuntu-latest)
- Python version (e.g. 3.11)
- Hash of **requirements.txt**

When the dependency file changes

- A new hash is generated.
- The previous cache is not used.
- A new cache is created with the new dependencies.

5. Cache Hit vs Cache Miss

Cache Hit (Faster)

- Dependencies are restored from cache.
- The install step is much faster.
- You will see a log message like:

```
Cache restored successfully
Cache hit: true
```

Cache Miss (Slower)

- No matching cache is found.
- Dependencies are downloaded and installed from the internet.
- You will see a log message like:

```
Cache not found for input keys
Cache hit: false
```

6. Check the Workflow Logs

1. Go to your repository on GitHub.
2. Click the **Actions** tab.
3. Open the latest workflow run.
4. Look at the **Set up Python** step.
5. Verify if the cache was restored or not.

```
Set up Python
▸ Run actions/setup-python@v5
  Cache restored successfully
  Cache hit: true
  Installed Python 3.11.6
▸ Run pip install -r requirements.txt
Requirement already satisfied ...
(faster install due to cache)
```

7. Security Reminder

- Do not store secrets, API keys or other sensitive data in caches.
- Caches are shared within your repository and can be accessed by future workflow runs.
- Only cache public dependencies (like Python packages).
- Use GitHub Secrets for sensitive information.
- Keep your dependencies file clean and free of sensitive data.

8. Example: Change a Dependency and See a Cache Miss

1. Add a new package to **requirements.txt** (e.g. **flask**).
2. Commit and push the change.
3. The workflow will run again.
4. Check the logs and you should see a cache miss.
5. Run the workflow again without changes and you should then see a cache hit.

9. Key Takeaways

- Caching speeds up your pipeline by reusing downloaded dependencies.
- **actions/setup-python** handles caching automatically.
- Cache keys depend on the OS, Python version and your dependency file.
- A change in **requirements.txt** creates a new cache.
- Never store secrets or sensitive data in caches.
- Always check the workflow logs to confirm cache restoration.

Separate Pipeline Work Into Multiple Jobs

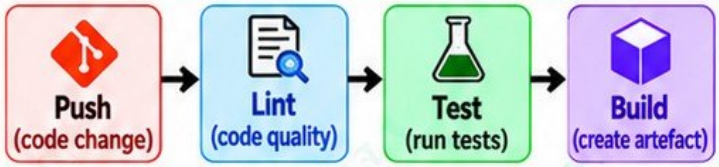
1. Why Use Multiple Jobs?

- Teams separate code quality checks, tests and build tasks to make the pipeline faster, cleaner and easier to manage.
- Jobs can run in parallel or in sequence based on your needs.
- If a lint job fails, you can stop the pipeline early and avoid running later jobs.
- It makes it clear what each job is responsible for.



Smaller focused jobs lead to faster and more reliable pipelines.

2. Pipeline Flow: Push → Lint → Test → Build



3. Multiple Jobs in a Workflow

.github/workflows/ci.yml

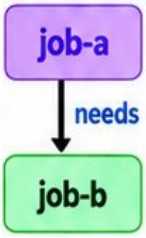
```yaml
name: Python CI
on: [push]
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - run: pip install ruff
      - run: ruff check .

  test:
    needs: lint  # runs after lint is successful
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - run: pip install -r requirements.txt
      - run: python -m pytest -q

  build:
    needs: test  # runs after test is successful
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build application
        run: echo "Building application..."
```

4. Understanding the needs Keyword

- needs controls the order of jobs.
- A job with needs will only run if the required job completes successfully.
- Without needs, jobs run independently (in parallel by default!).
- Use needs when one job depends on the output or success of another job.



5. Independent Jobs vs Dependent Jobs

Independent Jobs (run in parallel)

- No needs keyword.
- Jobs start at the same time.
- Useful for tasks that do not depend on each other.



Run at the same time

Dependent Jobs (run in sequence)

- Use needs to control order.
- The next job waits for the previous job to complete.
- Useful when jobs depend on each other.



Run in sequence

6. Common Mistake

Jobs do not automatically share files.

- Each job runs in a fresh, clean environment.
- Files created in one job are not available in another job.
- To share files, you must use artifacts (actions/upload-artifact and actions/download-artifact!) or rebuild the files in the next job.



Each job has its own separate workspace.

7. Key Takeaways

- Use multiple jobs to separate code quality checks, tests and build tasks.
- Use needs to control the execution order.
- Jobs without needs run in parallel.
- Dependent jobs run only when the required job succeeds.
- Jobs do not share files automatically.
- Use artifacts if you need to pass files between jobs.
- This makes your pipeline faster, clearer and more maintainable.

Create and Upload Your First Build Artifact

1. What Is a Build Artifact?

- A build artifact is a file or folder created by your build process (e.g. a Python package).
- Teams preserve build outputs so they can be downloaded, tested, and deployed later.
- It ensures you always have the exact version that was built and tested.
- Artifacts are useful for deployments, releases and debugging.



Build artifacts preserve the output of your build process.

2. Package the Python Application

Create a simple Python package that can be distributed.

Build the package

```
# Create a source distribution (sdist)
python -m build --sdist
```

This creates a distributable package in the dist/ directory.

Project structure after build

```
project/
  app.py
  pyproject.toml
  dist/
    your-app-0.1.0.tar.gz
```

The dist/ folder contains the package file that will be uploaded as an artifact.

3. Use actions/upload-artifact

Add a step to upload the build artifact.

Upload artifact in GitHub Actions

```
- name: Upload build artifact
  uses: actions/upload-artifact@v4
  with:
    name: python-package
    path: dist/
    retention-days: 7
```



actions/upload-artifact

Uploads files from your workflow run so they can be downloaded later.

4. Artifact Name and Retention

- **name**: A custom name for the artifact (e.g. python-package).
- **path**: The file or folder to upload (e.g. dist/).
- **retention-days**: How long GitHub keeps the artifact (default is 90 days, but you can set a smaller or larger value).



Set the retention period based on how long you need to keep the artifact.

5. Download the Artifact from a Workflow Run

1. Go to your repository on GitHub.
2. Click the Actions tab.
3. Open the completed workflow run.
4. Scroll down to the Artifacts section and click the artifact name to download it.



6. Artifact vs Dependency Cache

Build Artifact

- Stores build outputs (e.g. packaged application).
- Intended for later use (download, deploy, release).
- Created and uploaded manually using actions/upload-artifact.
- Kept for a specific retention period.

Dependency Cache

- Stores downloaded dependencies (e.g. pip packages).
- Used to speed up future workflow runs.
- Managed automatically (e.g. by actions/setup-python).
- Not intended for deployment.
- Can be safely deleted and recreated.

7. Verify: Download and Inspect the Package

1. Download the artifact from the workflow run.
2. Extract the file locally.
3. Inspect the contents to confirm your package was created correctly.
4. Optionally, install and test the package.

```
# Extract the package
tar -xzf your-app-0.1.0.tar.gz

# Inspect contents
ls -R dist/

# (Optional) Install it
pip install your-app-0.1.0.tar.gz

# Test it
python -c "import app; print('OK')"
```

8. Key Takeaways

- Artifacts preserve your build outputs for later use.
- Use actions/upload-artifact to upload files.
- Set a clear name and retention period.
- Download artifacts from the Actions tab.
- Artifacts are different from dependency caches.
- Verifying the downloaded package ensures the build produced the expected output.

Understand Secrets, Tokens, and Workflow Permissions

1. Why CI Pipelines Need Credentials?

- CI pipelines sometimes need credentials to:
  - Access private repositories.
  - Deploy applications.
  - Push packages to a registry.
  - Use cloud services (e.g. AWS, Azure).
  - Call external APIs.
- Credentials allow the workflow to perform these tasks securely.



Credentials give the pipeline access to protected resources.

2. GitHub Actions Secrets and Tokens

Actions Secrets

- Secrets store sensitive information (e.g. API keys, cloud credentials).
- They are encrypted and only available to workflows.
- You create them in: Settings → Secrets and variables → Actions.
- Use them in your workflow as: `${{ secrets.MY_SECRET }}`.

GITHUB_TOKEN

- A built-in token provided automatically by GitHub Actions.
- Used to authenticate the workflow with the repository.
- Has limited permissions based on your workflow settings.
- No need to create or store it as a secret.

3. Job Permissions Control Repository Access

- Job permissions define what the GITHUB_TOKEN can do in your workflow.
- By default, GitHub gives a set of permissions, but you can reduce them for better security.
- Use the permissions key to specify only what your workflow needs.
- This follows the principle of least privilege (only give the minimum access required).



Limit permissions to reduce risk and keep your repository safe.

4. Example: Read-Only Permissions for CI

This workflow uses read-only access to the repository contents.

.github/workflows/ci.yml

```yaml
name: Python CI
on: [push, pull_request]
permissions:
  contents: read   # read-only access
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run tests
        run: python -m pytest -q
```

contents: read allows the workflow to check out and read your code but cannot push changes to the repository.

5. Using a Secret in a Workflow

Example: Using an API key stored as a secret.

```yaml
- name: Call an external API
  env:
    API_KEY: ${{ secrets.MY_API_KEY }}
  run: |
    curl -H "Authorization: Bearer $API_KEY" \
     https://api.example.com/data
```

The actual secret value is hidden in the logs. Never print the secret or hardcode it in your YAML file.

6. Do Not Expose Secrets in Logs

- Never print secrets in logs (e.g. `echo $API_KEY`).
- Do not hardcode sensitive values in your YAML file.
- Secrets can be exposed in logs and accessed by others.
- Use GitHub Actions secrets instead.
- If you need to debug, remove or mask sensitive information.

7. Be Careful with Pull Requests from Untrusted Contributors

- Pull requests from people outside your trusted team can run your workflows.
- They might try to access secrets or modify the workflow.
- Do not give sensitive access (secrets, write permissions) to workflows triggered by untrusted contributors.
- Use read-only permissions for pull_request events.
- Review changes carefully before merging.



Untrusted contributors should not have access to your secrets or write permissions.

8. Key Takeaways

- Use secrets for sensitive information.
- GITHUB_TOKEN is provided automatically.
- Set job permissions to the minimum required (e.g. contents: read).
- Never print secrets or hardcode them in YAML.
- Be careful with pull requests from untrusted contributors.
- Follow the principle of least privilege to keep your repository and credentials safe.

Run CI Automatically on Pull Requests

1. Why Check Code Before Merging?

- Pull requests let multiple developers work on the project safely.
- Running CI on pull requests checks the code before it is merged.
- It helps catch problems early (lint issues, test failures, build errors).
- It keeps the main branch stable and production-ready.

CI on pull requests helps prevent broken code from being merged.

2. Configure the Workflow to Run on pull_request

Add the pull_request trigger to your workflow file.

.github/workflows/ci.yml

```yaml
name: Python CI
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
```

This runs the same CI checks for both pushes and pull requests targeting the main branch.

3. How a Pull Request Gets Automated Feedback

Feature Branch (make changes) → Pull Request (open on GitHub) → CI Checks (run automatically) → Review (humans review code) → Merge (when approved)

4. Example: Pull Request Checks in GitHub

Add new feature

Open #3 feature-branch → main

Conversation 0
Commits 1
Checks 3
Files changed 2

- Python CI / lint (pull_request) Successful in 24s
- Python CI / test (pull_request) Successful in 32s
- Python CI / build (pull_request) Successful in 18s

5. Passing Checks Does Not Always Mean Safe to Merge

- Passing checks only means the code passed the automated tests and checks.
- It does not guarantee the code is free from security issues, logic errors or business risks.
- Human review is still required.
- Check for code quality, security, design, and whether the change meets project requirements.

Passing CI checks is necessary but not sufficient for a safe merge.

6. Practice: Open a Pull Request and Inspect Checks

1. Create a new branch and make a small change.

```
git checkout -b feature/add-message
# make a change in your code
git add .
git commit -m "Add a new message"
git push -u origin feature/add-message
```



2. Open a pull request on GitHub.
3. Wait for the CI checks to run.
4. Click the Checks tab to view the results.
5. Fix any failing checks and push new commits.

Each new commit will automatically trigger the CI checks again.

7. Complete Pull Request Workflow

1. Developer pushes code to a feature branch.
2. Developer opens a pull request.
3. CI runs automatically on the pull request.
4. Developer checks the CI results.
5. Team reviews the code.
6. Fix issues if any checks fail.
7. Merge the pull request after approval.

8. Key Takeaways

- Run CI on pull_request to check code before merging.
- Pull requests provide automatet feedback.
- Passing checks do not automatically make the code safe.
- Human review is still required.
- Use this workflow to keep your main branch stable and reliable.

Protect Your Main Branch With Required Checks

1. Why Protect the Main Branch?

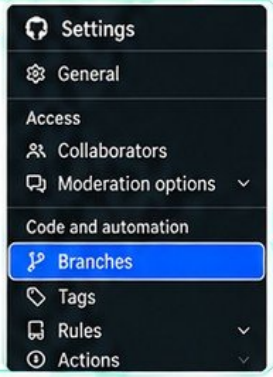
- Merging **untested or unreviewed code** can introduce **bugs, security issues** and break the application.
- It can cause **downtime, data loss** or **rollbacks**.
- Branch protection helps ensure only tested and approved code reaches the main branch.



Protection reduces risk and keeps your main branch stable and safe.

2. Branch Protection Rules and Rulesets

- GitHub provides **branch protection rules** and **repository rulesets** to control how code is merged.
- You can apply rules to the **main** branch (e.g. `main`) or use rulesets for more flexible policies.
- Rules can require **status checks, reviews, prevent direct pushes** and restrict who can merge.

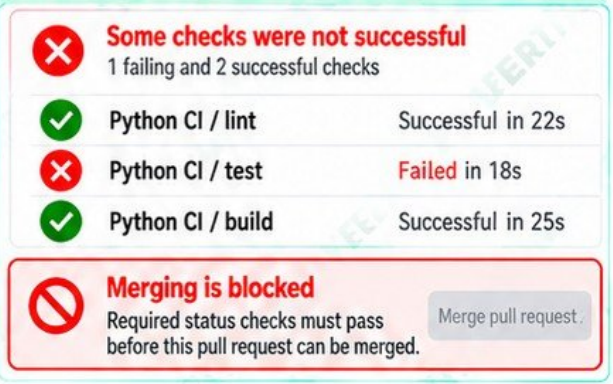


3. Configure Required Status Checks

1. Go to **Settings → Branches**.
2. Click **Add rule** or create a ruleset.
3. Select your **main** branch (e.g. `main`).
4. Enable **Require status checks to pass before merging.**
5. Select the required checks (e.g. lint, test, build).



4. What Happens When a Check Fails?



5. Stable Job Names Are Important

- The name of the status check comes from your workflow **job name**.
- If you change the job name, you must update the required check in the branch rule.
- Use clear and **stable** job names (e.g. lint, test, build) to avoid confusion.

```yaml
jobs:
  lint:
    name: lint  # stable name
    ...
  test:
    name: test  # stable name
    ...
  build:
    name: build # stable name
...
```

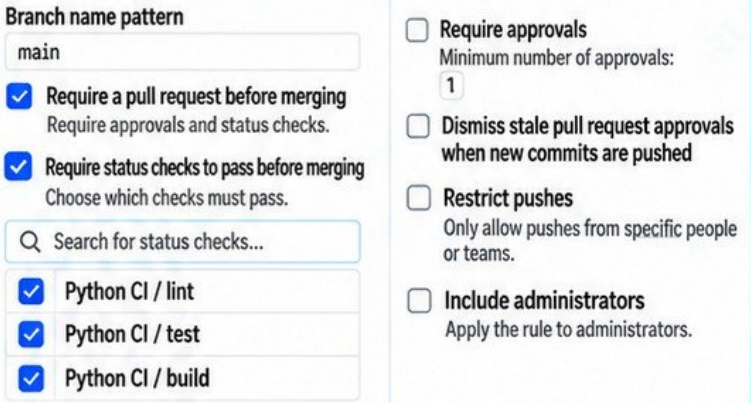
6. Peer Review and Least Privilege

- Require **pull request reviews** before merging.
- Use code reviews to catch logic errors, security issues and design problems.
- Only give write/merge permissions to trusted team members (principle of **least privilege**).
- This ensures changes are reviewed and approved by the right people.



Reviews + limited permissions make your main branch more secure and reliable.

7. Example: Branch Protection Settings



8. Important Notes

- The availability of some repository rules and features depends on your GitHub plan and whether the repository is public or private.
- Rulesets provide more flexible and advanced options than classic branch protection rules.
- Branch protection does not replace good code review, testing and security practices.

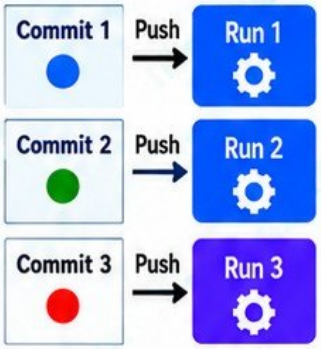
9. Key Takeaways

- Protect the main branch to prevent untested or unreviewed code from being merged.
- Use required status checks and pull request reviews.
- Keep job names stable.
- Apply the principle of least privilege.
- Repository rules and rulesets availability depends on your GitHub plan and repository visibility.

Prevent Duplicate Runs and Unsafe Pipeline Behavior

1. Why Duplicate Runs Happen?

- Each push to your repository can trigger a new workflow run.
- If you push multiple times quickly, you can have multiple runs running at the same time.
- This can waste resources, make results confusing, and deploy outdated code.
- It is common during active development when you push several changes in a short time.



Multiple pushes can trigger multiple workflow runs that run at the same time.

2. Use Concurrency Groups

- Concurrency groups ensure that only one run for a specific branch or workflow is active.
- When a new run starts, GitHub can cancel the previous run in the same group.
- This prevents duplicate runs and keeps your pipeline results clear.

```yaml
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Run tests
        run: python -m pytest -q
```

Only the latest run is kept. Previous runs in the same group are cancelled.

3. How cancel-in-progress Works

- cancel-in-progress: true cancels any previous run in the same concurrency group when a new run starts.
- This is useful for CI checks where you only care about the latest code.
- It saves time and resources.



4. When to Use or Avoid Cancellation

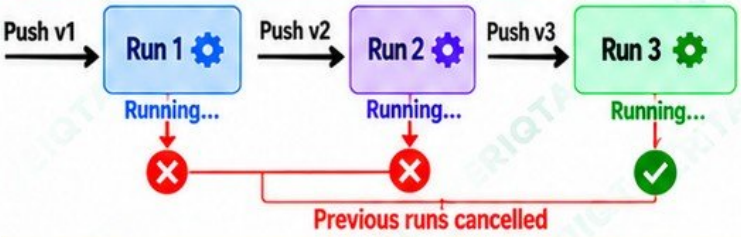
Good for Cancellation

- CI checks for pull requests and feature branches.
- Linting, testing and build jobs.
- Situations where only the latest result matters.
- Saves time and compute resources.

Be Careful or Avoid

- Production deployments (do not cancel in-progress).
- Long-running jobs that should always complete.
- Jobs that create important external changes.
- Use cancellation only when it is safe to stop the previous run.

5. Why Cancellation Is Useful



Only the latest code is tested, so you get faster and clearer results.

6. Third-Party Action Trust and Pinning

- Your workflow often uses third-party actions (e.g. actions/checkout, setup-python).
- Use trusted actions from reputable sources.
- Pin actions to a specific commit SHA instead of a moving tag (e.g. @v4) to avoid unexpected changes.
- Regularly review and update action versions.

```yaml
# Using a specific commit SHA (example)
- uses: actions/checkout@3df4f2c8d4e6a7b9c...

# Not recommended (moving tag can change)
- uses: actions/checkout@v4
```

Pin to a specific commit SHA for more security and predictability.

7. Production Awareness



- Automation must be predictable, not just fast.
- Ensure your pipeline behaviour is clear and consistent.
- Use cancellation and concurrency carefully based on the job type.
- Always consider the impact on deployments and external systems.
- Test changes to your workflow configuration before using it in production.

8. Key Takeaways

- Multiple pushes can trigger overlapping workflow runs.
- Use concurrency groups to prevent duplicate runs.
- cancel-in-progress is useful for CI but can be risky for deployments.
- Use trusted third-party actions and pin to specific commit SHAs.
- Automation must be predictable, not merely fast.
- Choose cancellation settings based on the job type and risk.

Troubleshoot Real GitHub Actions Failures

1. Investigation Sequence

1	Trigger	Identify what started the workflow (push, pull request, schedule).
2	Job	Open the failed job.
3	Step	Find the step that failed.
4	Logs	Read the error messages and context.
5	Root Cause	Determine the real reason for the failure.
6	Fix	Update the code or configuration.
7	Verify	Rerun the workflow and confirm it works.



2. ModuleNotFoundError (Missing Dependencies)

This error happens when required Python packages are not installed in the workflow environment.

```
ModuleNotFoundError: No module named 'requests'
  File "app.py", line 1, in <module>
    import requests
```

Common Causes

- Dependencies not installed.
- requirements.txt is missing or incomplete.
- Installing in the wrong environment.

Fix

- Add a step to install dependencies.
- Check that requirements.txt contains all required packages.
- Use the correct Python version.

```
- name: Install dependencies
  run: pip install -r requirements.txt
```

3. Incorrect File Paths and Working Directory

The workflow may run from a different directory than expected.

```
python: can't open file '/home/runner/work/project/app.py':
  [Errno 2] No such file or directory
```

Common Causes

- Wrong file path.
- Running the command from the wrong directory.
- File not checked into the repository.

Fix

- Check the repository structure.
- Use the correct path in your command.
- Set the working directory if needed.

```
- name: Run script
  working-directory: ./app
  run: python app.py
```

4. YAML Indentation and Configuration Errors

YAML is sensitive to indentation and structure.

```
Error: .github/workflows/ci.yml (Line 12, Column 3):
    The mapping values are not allowed in this context.
```

Common Causes

- Incorrect indentation.
- Missing or misplaced keys.
- Invalid YAML structure.
- Using tabs instead of spaces.

Fix

- Check indentation (use spaces).
- Validate the YAML structure.
- Use a YAML linter or GitHub's error messages to find the line with the issue.

5. Permission Failures

The workflow does not have the required permissions.

```
Error: Resource not accessible by integration
       or permission denied
```

Common Causes

- Missing or insufficient job permissions.
- Trying to access restricted resources (e.g. packages, deployments).
- Using GITHUB_TOKEN without required permissions.

Fix

- Add the required permissions in your workflow.
- Use the principle of least privilege.

```
permissions:
  contents: read
  packages: write  # only if needed
```

6. Missing Artifacts and Unavailable Secrets

Missing Artifacts

```
Error: No artifact found
with the name 'build'
```

Common Causes

- Artifact not uploaded.
- Wrong artifact name.
- Artifact from a different job.

Fix

- Check the upload step.
- Use the correct artifact name.
- Ensure the job that creates it ran successfully.

Unavailable Secrets

```
Error: secret not found or
empty value
```

Common Causes

- Secret not created in repository.
- Using the wrong secret name.
- Secrets are not available for forked pull requests.

Fix

- Add the secret in Settings → Secrets and variables → Actions.
- Check the secret name in your YAML.
- Use appropriate permissions and understand fork limitations.

7. Practice: Diagnose a Broken Workflow

1. Open the provided broken workflow run.
2. Identify the failure using the investigation sequence (Trigger → Job → Step → Logs → Root Cause).
3. Fix the issue in the code or workflow file.
4. Push the change and rerun the workflow.
5. Confirm that all checks pass.



Use the error message and logs to find the real cause, not guess.

8. Key Takeaways

- Follow a clear investigation sequence.
- Read the actual error messages and logs.
- Check dependencies, file paths, YAML structure, permissions, artifacts and secrets.
- Use evidence to find the root cause.
- Fix the issue and verify the workflow.
- Workflows should be predictable and consistent.

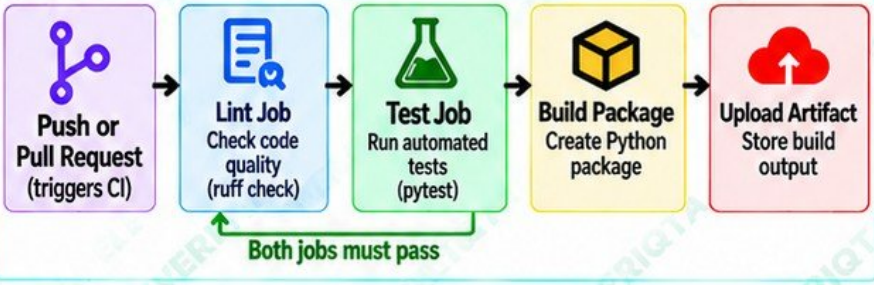
Final Project: Build a Complete GitHub Actions CI Pipeline

1. Project Overview

- In this final project, you will bring together everything from the previous 19 pages.
- You will build a complete CI pipeline for a Python project that:
  - Runs on push and pull_request.
  - Checks out the repository.
  - Sets up the required Python version.
  - Installs and caches dependencies.
  - Runs code quality checks (Ruff).
  - Executes automated tests.
  - Builds a Python package.
  - Uploads the package as an artifact.
  - Uses minimum necessary permissions.
  - Displays meaningful success or failure results.

A complete CI pipeline that checks, tests, builds and produces a usable artifact.

2. Pipeline Workflow Diagram



3. Complete GitHub Actions Workflow (.github/workflows/ci.yml)

```yaml
name: Python CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

permissions:
  contents: read

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  lint:
    name: Lint (ruff)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
          cache: 'pip'
      - run: pip install -r requirements.txt
      - run: ruff check .

  test:
    name: Test (pytest)
    runs-on: ubuntu-latest
    needs: lint
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
          cache: 'pip'
      - run: pip install -r requirements.txt
      - run: pytest -v

  build:
    name: Build Package
    runs-on: ubuntu-latest
    needs: [lint, test]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
          cache: 'pip'
      - run: pip install -r requirements.txt
      - run: python -m build
      - uses: actions/upload-artifact@v4
        with:
          name: python-package
          path: dist/
          retention-days: 7
```

4. What Each Job Does

Lint Job (ruff)

- Checks code quality.
- Finds common issues early.
- Fails if there are linting errors.

Test Job (pytest)

- Runs automated tests.
- Ensures the code works.
- Fails if any test fails.

Build Package

- Creates a Python package (e.g. .whl and .tar.gz).
- Runs only if lint and test pass.
- Uploads the package as an artifact for download.

5. Successful Workflow Run

Python CI
succeeded 2 minutes ago

lint	18s
test	32s
build	28s

All jobs completed successfully.
Your code passed linting, tests and the package was built and uploaded.

6. Failed Workflow Run (Example)

Python CI
failed 1 minute ago

lint	16s
test	14s
build	0s

The test job failed and the build job did not run. Check the logs, fix the issue and run again.

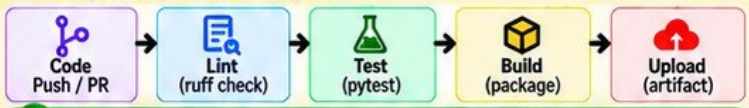
7. Practice: Verify and Troubleshoot

1. Push your code to run the CI pipeline.
2. Open a pull request to run the pipeline again.
3. Confirm that lint, test and build jobs pass.
4. Intentionally introduce a failing test (e.g. assert 1 == 2).
5. Push the change and observe the failed run.
6. Investigate the logs to find the error.
7. Fix the test and push again.
8. Confirm that all jobs pass and the artifact is created.

8. Key Takeaways

- You built a complete CI pipeline that combines all the key concepts.
- The pipeline runs on push and pull requests.
- It uses separate lint and test jobs, and a dependent build job.
- It installs and caches dependencies for faster runs.
- It uploads a build artifact that can be downloaded.
- It uses minimum necessary permissions for security.
- You learned how to investigate failures using logs and fix real issues.
- A reliable CI pipeline gives fast, clear feedback and keeps your project healthy.

Final Pipeline Flow (Conceptual)



Final Result: A complete, working CI pipeline with clear success or failure feedback.